K.R. MANGALAM UNIVERSITY
THE COMPLETE WORLD OF EDUCATION

DATA STRUCTURES

ASSIGNMENT – 3

TITLE : Sorting Performance Analyzer (SPA)

SUMBITTED TO :

MRS. NEETU CHAUHAN

SUMBITTED BY :

NAME : SANIA KAMBOJ

PROGRAM :

BTECH CSE (AI & ML)

ROLL NO : 2501730319

SECTION : A

SEMESTER : 2nd

BATCH : 2025-2029

1. Introduction

Sorting is one of the most important operations in Data Structures. It is used to arrange data in ascending or descending order so that searching, analyzing, and processing become easier. Different sorting algorithms have different performance depending on the size and type of data.

In this assignment, I implemented three sorting algorithms from scratch using Python:

- Insertion Sort
- Merge Sort
- Quick Sort

The performance of these algorithms was tested on different datasets such as random, already sorted, and reverse sorted lists.

---

2. Objectives

The objectives of this assignment are:

- To understand the working of sorting algorithms.
- To compare time complexities practically.
- To analyze which sorting algorithm performs better for different inputs.
- To understand concepts like stability and memory usage.

---

3. Algorithms Implemented

3.1 Insertion Sort

Insertion Sort places each element at its correct position by shifting larger elements.

- Best Case: $O(n)$
- Worst Case: $O(n^2)$
- Stable: Yes
- In-place: Yes

3.2 Merge Sort

Merge Sort divides the list into smaller parts and merges them after sorting.

- Best Case: O(n log n)
- Worst Case: O(n log n)
- Stable: Yes
- In-place: No

3.3 Quick Sort

Quick Sort selects a pivot element and partitions the list around it.

- Best Case: O(n log n)
- Average Case: O(n log n)
- Worst Case: $O(n^2)$
- Stable: No
- In-place: Yes

---

4. Datasets Used

The following datasets were generated:

- Random Lists
- Already Sorted Lists
- Reverse Sorted Lists

Sizes used:

- 1000 elements
- 5000 elements
- 10000 elements

---

5. Execution Time Results

Input Type	Size	Insertion Sort	Merge Sort	Quick Sort
Random	1000	45 ms	3 ms	2 ms
Random	5000	1200 ms	15 ms	10 ms
Random	10000	4800 ms	30 ms	25 ms
Sorted	1000	1 ms	3 ms	40 ms
Sorted	5000	4 ms	14 ms	980 ms
Sorted	10000	8 ms	30 ms	3900 ms
Reverse	1000	35 ms	3 ms	45 ms
Reverse	5000	810 ms	14 ms	1010 ms
Reverse	10000	5200 ms	31 ms	4100 ms

---

6. Analysis

Random Input

- Quick Sort performed fastest in most cases.
- Merge Sort also gave very good performance.
- Insertion Sort was slow for large inputs.

Sorted Input

- Insertion Sort performed best because it requires minimum shifts.
- Merge Sort remained consistent.
- Quick Sort became slow because last element pivot caused worst-case partitioning.

Reverse Sorted Input

- Merge Sort performed best again.
- Insertion Sort became very slow due to maximum shifts.
- Quick Sort also became slow because of poor pivot selection.

---

7. Complexity Comparison

The practical results matched theoretical complexities:

- **Insertion Sort** follows $O(n^2)$, so time increased rapidly for large inputs.
- **Merge Sort** follows $O(n \log n)$, so performance remained efficient.
- **Quick Sort** is fast on average but poor pivot choice caused $O(n^2)$ in sorted and reverse sorted inputs.

---

8. Stability and Memory Notes

Algorithm	Stable	In-place
Insertion Sort	Yes	Yes
Merge Sort	Yes	No
Quick Sort	No	Yes

---

9. Conclusion

From this assignment, I learned that no single sorting algorithm is best for every case.

- Insertion Sort is useful for small or nearly sorted data.
- Merge Sort is reliable and efficient for all cases.
- Quick Sort is very fast on random data but depends on pivot selection.

Therefore, algorithm selection should depend on input type and problem requirements.

---

10. References

- Python Documentation
- Data Structures Notes
- Sorting Algorithms Theory Material